

Design Optimization & Scalability Techniques

35 Named Industry Patterns Used in the RMC

From CAM/RAM separation to multi-core scaling discipline

Scope	Every reusable hardware-design and scalability pattern actually used in the RMC, named and explained
Part I	15 block-level design techniques (storage, hazards, arbitration, ownership)
Part II	20 scalability techniques (growth from 1 to N cores, clients, configurations)
Format	Per technique: industry name, mechanism, why it beats the naive alternative, where it's used here
Audience	Anyone reviewing the RMC who wants the standard vocabulary for what's already been built

Contents

Part I: Block-Level Design Techniques	4
1. CAM/RAM Split (Tag/Data Array Separation)	4
0.1 The Problem	4
0.2 The Technique	4
2. Associativity (Fully-Associative vs. Set-Associative vs. Direct-Mapped Storage)	5
0.3 The Three Options	5
0.4 Why Fully-Associative Is the Right Default for In-Flight Request Pools	5
3. Timestamp/Deadline Scoreboarding (Absolute-Deadline Fields with Modular Sequence Numbering)	6
0.5 The Naive Alternative: One Counter Per Constraint	6
0.6 The Technique	6
0.6.1 The Wraparound Problem and Its Standard Fix	6
4. FSM Table Decoupling (Scoreboard / Control Separation)	7
0.7 The Naive Alternative: State Inside the Controller	7
0.8 The Technique	7
5. Single-Writer Principle (Single-Writer, Multiple-Reader (SWMR) Discipline)	8
0.9 The Problem	8
0.10 The Technique	8
6. Leaky-Bucket Accumulator (Leaky-Bucket Algorithm)	9
0.11 The Technique	9
7. Watermark Hysteresis (Schmitt-Trigger Pattern (Dual-Threshold Debouncing))	10
0.12 The Technique	10
8. Sliding Window Rate Limiter (Sliding Window Counter)	11
0.13 The Technique	11
9. Store-to-Load Forwarding (Pipeline Hazard Forwarding / Bypassing)	12
0.14 The Technique	12
10. Cheap-Filter-Then-Expensive-Verify (Two-Stage Filtering)	13
0.15 The Technique	13
11. Admission Control via Reservation (Resource Reservation (vs. Issue-and-Retry))	14
0.16 The Two Strategies	14
12. Age-Based Anti-Starvation (Aging Counters (Fairness Arbitration))	15
0.17 The Technique	15
13. Shadow / Double-Buffered Configuration Registers (Shadow Register Pattern)	16
0.18 The Problem	16
0.19 The Technique	16
14. Priority Encoder / Reduction Tree (Parallel Reduction (Priority Encoding, Argmax Tree))	17
0.20 The Technique	17

15. Credit-Based Flow Control (Credit-Based Flow Control (vs. Valid/Ready Handshake))	18
0.21 The Two Standard Handshake Models	18
Part II: Scalability Techniques	19
16. Parameterized RTL (Configuration-Driven Width Parameterization)	20
0.22 The Technique	20
17. Modular / Hierarchical Instantiation (Replicate-the-Core Pattern)	21
0.23 The Technique	21
18. Address-Space Partitioning at the Routing Layer (Decode-Above-the-Unit Pattern)	22
0.24 The Technique	22
19. Interface Width Over-Provisioning (Forward-Sized Interfaces)	23
0.25 The Technique	23
20. Shared Resource Pooling (Pooled vs. Per-Client Allocation)	24
0.26 The Technique	24
21. Fairness-Based Pool Sharing (Aging in Place of Static Reservation)	25
0.27 The Technique	25
22. Throttled Admission (Meter-Don't-Burst Pattern)	26
0.28 The Technique	26
23. Single Clock-Domain-Crossing Boundary (Centralized CDC)	27
0.29 The Technique	27
24. Bounded Associativity (Throttle-the-Input Before You Grow-the-CAM)	28
0.30 The Technique	28
25. Peer-FSM Extensibility (Add-a-Peer, Not Add-a-Special-Case)	29
0.31 The Technique	29
26. Single Global Timebase (One Clock Reference, Many Comparators)	30
0.32 The Technique	30
27. Tiered Hierarchical Arbitration (Staged Arbitration, Not Flat N-Way)	31
0.33 The Technique	31
28. Rate-Decoupling Buffers (Buffer-the-Mismatch, Not Match-the-Clocks)	32
0.34 The Technique	32
29. Configuration-Time Capability Negotiation (Init-Time Profile Selection, Not Compile-Time Branching)	33
0.35 The Technique	33
30. Depth-Bounded Graceful Degradation (Bound the Resource, Let Logic Absorb Pressure)	34
0.36 The Technique	34
31. Event-Driven Flag Propagation (Flat Flag Reads, Not Deep Query Chains)	35
0.37 The Technique	35

32. Symmetric Per-Unit Design (No Special-Cased “Unit Zero”)	36
0.38 The Technique	36
33. Explicit Interface Contracts (Documented Boundaries Over Defensive Logic)	37
0.39 The Technique	37
34. Generalized Reservation-Before-Commitment (Check-Then-Commit at Every Scaling Boundary)	38
0.40 The Technique	38
35. Self-Describing Configuration (Explicit Version/Width Fields Over Inferred Structure)	39
0.41 The Technique	39

Part I: Block-Level Design Techniques

The techniques below describe how individual blocks and structures inside the RMC are built — storage organization, hazard handling, arbitration, ownership discipline.

Technique 1: CAM/RAM Split

Industry name: Tag/Data Array Separation

The Problem

A request buffer entry has two very different kinds of fields: a small set of *search keys* (bank group, bank, row, column — what you compare an incoming request against) and a much larger *payload* (512-bit write data, status bits, buffer pointers). If you build the buffer as one uniform structure and make every field searchable, you pay CAM-cell cost (XOR-compare circuitry on every bit, roughly $1.5\text{--}2.5\times$ the transistor count of a plain SRAM cell) for fields that never actually need to be searched — the 512-bit data payload, for instance, is never compared against anything; it only ever needs to be *read out* once you already know which entry matched.

The Technique

Split the structure into two physically separate arrays:

- A **tag array** (true CAM): holds only the searchable key fields, with comparator logic on every cell, evaluated against all entries in parallel every cycle.
- A **data array** (plain indexed SRAM): holds the payload, with a single read port, addressed by the index the tag array's match logic produces.

This is exactly how every set-associative cache is built: the tag array (comparators, one set per way) is physically separate from the data array (plain SRAM, read out only after the tag array says which way hit).

Used here: the Write Request Buffer (32–64 entries, holds `bank_group`, `bank`, `row`, `column` — the search keys for RAW Redirect and Stage 2's bank scan) is kept separate from the Write Data Buffer (32–64 entry SRAM, 576 bits/entry, holds the actual write data + byte mask, addressed by `data_buf_idx`). The Write Data Buffer is never searched — it's read out exactly once, after the Write Request Buffer's CAM compare has already produced a matching index.

Why this beats the naive alternative: if data and tags lived in one CAM-everything structure, every comparator in the array would need to span the full entry width (search key + 512-bit payload), even though the payload bits never participate in any compare. Splitting means the expensive CAM cells exist only for the ~ 22 bits that are actually searched; the remaining ~ 576 bits live in cheap, single-ported SRAM. For a 32-entry buffer this is the difference between a 32×598 -bit CAM and a 32×22 -bit CAM plus a 32×576 -bit plain SRAM — the latter is dramatically smaller.

Technique 2: Associativity

Industry name: Fully-Associative vs. Set-Associative vs. Direct-Mapped Storage

The Three Options

Any structure that maps a search key to a stored entry has to choose how many possible *locations* a given key could land in:

Scheme	Mechanism	Cost / Tradeoff
Direct-mapped	Key hashes to exactly one slot. No comparator needed — just index and read.	Cheapest hardware. Guarantees collisions: two different keys that hash to the same slot evict each other even if other slots are empty.
Set-associative (N -way)	Key hashes to one of K sets; compare against the N entries within that set only.	Middle ground. Reduces comparator count to N (not full table depth) at the cost of possible conflicts <i>within</i> a set even when the whole table has room. Standard choice for CPU caches (4-way, 8-way L1/L2).
Fully-associative	Key compared against <i>every</i> entry in the table, simultaneously.	Most flexible — any entry can hold any key, zero hash-induced collisions. Most expensive: comparator count equals full table depth.

Why Fully-Associative Is the Right Default for In-Flight Request Pools

Caches use set-associativity because they're exploiting *address locality* — nearby addresses being accessed together is the whole point of a cache, and that locality is exactly what makes hashing-into- sets a reasonable trade (you accept some conflict risk in exchange for far fewer comparators). A request buffer or RAW-hazard CAM has **no such locality to exploit**: an incoming read could legitimately need to be checked against *any* currently-outstanding write, regardless of how recently that write was issued or what address pattern preceded it. There is nothing to hash on that predicts which entries are worth comparing against and which aren't.

Used here: the RAW Redirect CAM and both Write/Read Request Buffers are fully-associative by necessity, not by default choice. Any pending read must be checkable against *all* valid write entries — there's no address-locality assumption that would let a set-associative scheme safely ignore some entries. Trying to hash into sets here would add complexity (a hash function, set-index logic) while providing zero benefit, since the workload has no locality property for that hash to exploit.

Where set-associativity *would* apply in this design, if ever built: a future TLB-like structure or page-table-walk cache inside the MC *would* have genuine address locality to exploit (programs exhibit spatial/temporal locality in their access patterns), and a 4-way or 8-way set-associative cache would be the standard choice there — unlike the RAW-redirect CAM, which is correctly fully-associative because in-flight transaction pools don't have a comparable locality property.

Technique 3: Timestamp/Deadline Scoreboarding

Industry name: Absolute-Deadline Fields with Modular Sequence Numbering

The Naive Alternative: One Counter Per Constraint

The obvious way to enforce a timing constraint (“bank b cannot accept an ACT for the next N cycles”) is a down-counter: load it with N at the triggering event, decrement every cycle, gate the command on “counter == 0”. For a single bank with a single constraint this is cheap. For 16 banks $\times$ roughly a dozen JEDEC timing parameters each, this naive approach needs **on the order of 200 independent live decrementers**, each with its own subtractor, running every cycle regardless of whether that constraint is even close to expiring.

The Technique

Replace every countdown with a single stored value: the **absolute future cycle** at which the constraint clears, computed once at issue time as `deadline = current_cycle + delay`. Checking eligibility later is a single comparison: `current_cycle >= deadline`. The arithmetic (the addition) happens rarely — once per command issued. The comparison (cheap) happens every cycle, but a comparator is significantly less hardware than a live subtractor-and-register pair maintained every cycle regardless of relevance.

The Wraparound Problem and Its Standard Fix

A free-running cycle counter eventually wraps. A naive `current_cycle >= deadline` comparison breaks exactly at the wrap boundary. The standard fix — used for exactly this reason in TCP sequence numbers — is **modular (signed-difference) comparison**:

$$\text{diff} = \text{current_cycle} - \text{deadline} \pmod{2^W} \quad \text{eligible} = (\text{diff}_{[W-1]} = 0)$$

where W is the counter width and bit $W-1$ is treated as a sign bit over a 2^W -wide modular space. As long as the true gap between `current_cycle` and any live `deadline` never exceeds half the counter’s range, this subtraction-and-sign-check is wrap-safe by construction — the same trick that lets TCP sequence numbers wrap every few gigabytes of traffic without ever breaking ordering logic.

Used here: every `next_act`, `next_pre`, `next_cas`, `next_ref` field in the Bank FSM Table (and the shared cross-bank fields like `next_act_same_bg`, `faw_window`) is an absolute deadline against a single shared `global_cycle` counter, sized at 20 bits. The width was chosen by checking the worst-case gap ($8 \times \text{tREFI}$ postponement, $\approx 12,480$ cycles) against the wraparound margin (half of $2^{20} = 524,288$ cycles) — a $24\times$ safety margin, versus a 15-bit counter’s margin of only $\approx 1.3\times$, which was rejected as too tight.

Why this beats N live counters: roughly 100 deadline fields plus 100 static comparators against one shared counter replaces $\approx 200+$ independent decrementing counters with their own subtractors. Comparators are simpler hardware than maintained countdowns, and centralizing the “what time is it” question into one counter means every constraint shares the same clock reference instead of each tracking its own relative countdown.

Technique 4: FSM Table Decoupling

Industry name: Scoreboard / Control Separation

The Naive Alternative: State Inside the Controller

A common first-pass design embeds bank state directly inside the scheduler/controller logic — the controller *is* the state machine for every bank, with state transition logic, timing checks, and command-issue decisions all interleaved in one block. This makes the controller large, hard to verify in isolation, and forces every new bank or every new constraint to touch the same monolithic logic block.

The Technique

Separate *where state lives* from *what decides based on that state*. A dedicated table (the Bank FSM Table) is the sole owner of per-bank state and timing fields. The controller (Scheduler) is reduced to a **reader**: every cycle, it scans the table's go/no-go bits and deadline fields, and the table's fields update by simple, mechanical rules triggered by command issue — not by controller-internal logic.

This is the same separation Tomasulo's algorithm uses between *reservation stations* (where operand/state tracking lives) and the *issue logic* (which only reads readiness bits to decide what to dispatch) — the issue logic doesn't recompute operand availability itself, it just checks a bit that the reservation station already maintains.

Used here: the Bank FSM Table holds `state`, `row_open`, and five go/no-go bits (`open_pending`, `pre_pending`, `act_pending`, `wr_pending`, `rd_pending`) per bank, plus the deadline fields from Technique 3. The Scheduler's Stage 2 go/no-go scan does not contain any bank-state transition logic itself — it purely reads these fields. State transitions (e.g. `ACTIVATING` → `BANK_ACTIVE` when `tRCD` expires) are mechanical updates triggered by the deadline fields crossing `global_cycle`, defined once in the field-update table, not re-derived inside the Scheduler.

Why this beats embedding state in the controller: the Scheduler can be verified against a simple contract (“given these go/no-go bits, pick the highest-priority eligible bank”) without needing to also model every JEDEC timing rule inline. The Bank FSM Table can be verified independently against the JEDEC timing spec, since its update rules are mechanical and don't depend on scheduling decisions. Changes to scheduling policy (Stage 3's tie-break rule, for instance) never risk breaking timing correctness, because timing correctness lives entirely in a different, untouched block.

Technique 5: Single-Writer Principle

Industry name: Single-Writer, Multiple-Reader (SWMR) Discipline

The Problem

Any piece of shared state — a config register, a status field, a go/no-go bit — becomes a race condition risk the moment more than one block can write it. If both the Maintenance Engine and the Scheduler could independently write a bank’s `state` field, for instance, a single cycle where both attempt an update produces an undefined result: whichever write “wins” depends on synthesis/place-and-route artifacts, not design intent.

The Technique

For every piece of shared state, designate exactly **one** block as the writer. Every other block that needs the information becomes a **reader only**. This is sometimes called Single-Writer Multiple-Reader (SWMR) discipline, standard practice in any concurrent system (hardware or software) where shared state exists.

Shared state	Sole writer	Readers
Bank FSM Table fields	Whichever block issued the triggering command (Scheduler issues ACT/RD/WR/PRE; Maintenance Engine issues REF/RFM)	Scheduler’s Stage 2 scan, RAW Redirect
<code>ref_pending</code> , <code>ref_urgent</code>	Maintenance Engine’s Refresh Scheduler sub-FSM only	Scheduler Stage 0
<code>global_cycle</code>	The counter itself (free-running, no external writer at all)	Every deadline comparison, system-wide
Config Registers (<code>tRCD</code> , <code>tRP</code> , etc.)	Host, via AXI4-Lite CSR path, <i>only while controller idle</i>	Every block that consumes timing parameters

Why this matters even when it seems obvious: the failure mode this prevents isn’t “two blocks write garbage simultaneously” (that’s caught immediately in simulation) — it’s the subtler case where two writers *usually* agree but occasionally race on a corner-case cycle, producing a bug that only manifests under specific timing conditions and is extremely hard to reproduce. Designating ownership up front, even when it currently “looks like” only one block would ever write a given field, removes an entire class of bugs before they can exist — it’s a discipline applied at specification time, not a fix applied after a race is found.

Used here: this principle is why Maintenance Engine was placed as a peer block writing into the *existing* Bank FSM Table fields (Model A from the architecture discussion) rather than maintaining its own separate copy of bank state. A separate copy would have created exactly the two-writer race this technique exists to prevent — the Scheduler’s copy and the Maintenance Engine’s copy could disagree about whether a bank is mid-refresh.

Technique 6: Leaky-Bucket Accumulator

Industry name: Leaky-Bucket Algorithm

The Technique

Borrowed from network traffic-shaping: a counter (the “bucket”) increments at a fixed rate and decrements whenever the corresponding action is taken. The bucket has a ceiling — once it reaches that ceiling, the system is forced to take the action immediately, regardless of anything else competing for priority. This lets a system *postpone* a periodic obligation under load, while guaranteeing it can never be postponed indefinitely.

Used here: the Refresh Scheduler’s credit tracker. Credits increment by 1 every tREFI interval and decrement by 1 each time a REF command is actually issued. JEDEC permits postponing refresh up to 8 intervals’ worth before it becomes mandatory — modeled directly as a bucket ceiling of 8. When credits reach 8, `ref_urgent` asserts and overrides every other priority level in the Arbitration Logic, forcing a REF through regardless of pending RD/WR traffic.

Why this beats a simple periodic interrupt: a naive “force a REF exactly every tREFI” design has no flexibility — it interrupts read/write traffic on a fixed schedule even when there’s a natural gap a few cycles later that would be cheaper to use. The leaky-bucket accumulator lets the Scheduler exploit short-term scheduling freedom (servicing a burst of row-hit reads first, for instance) while the bucket silently tracks how much slack remains, and only forces the issue once that slack is actually exhausted.

Technique 7: Watermark Hysteresis

Industry name: Schmitt-Trigger Pattern (Dual-Threshold Debouncing)

The Technique

When a binary mode needs to switch based on a fill level or other continuously-varying quantity, using a *single* threshold causes oscillation: a value hovering right at the threshold flips the mode back and forth every time it crosses, even by a tiny margin. The standard fix — identical in spirit to a Schmitt-trigger comparator in analog circuit design — is to use **two** thresholds: a high one to enter a mode, a lower one to exit it. The gap between them is a hysteresis band that absorbs small fluctuations without triggering a mode flip.

Used here: the Scheduler's RD/WR mode arbitration (Stage 1). `WR_HIGH_WM` (entry threshold, write-pool fill level that triggers a switch *into* WR_MODE) and `WR_LOW_WM` (exit threshold, switches *back* to RD_MODE) are deliberately set apart — not equal — specifically to prevent the mode from flipping every cycle when the write pool's fill level happens to sit near a single shared threshold.

Why a single threshold fails here: if entry and exit used the same watermark value, a write pool oscillating by ± 1 entry around that value (entirely plausible under steady traffic) would flip `mode_state` every cycle, and every flip costs a turnaround penalty (tWTR or tRTW, on the order of 10+ cycles per the pipeline walkthrough document). The hysteresis gap converts a potentially-every-cycle cost into a rare, well-amortized one.

Technique 8: Sliding Window Rate Limiter

Industry name: Sliding Window Counter

The Technique

To enforce “no more than K events within any rolling window of T time units,” keep a small ring buffer of the timestamps of the last K events. A new event is permitted only if the *oldest* entry in the ring buffer is already further than T in the past — i.e., the window has rolled past it. This is the same technique used by API rate-limiters (“100 requests per rolling minute”) and is cheaper than maintaining a full histogram of recent activity.

Used here: the Bank FSM Table’s `faw_window[3:0]` field — a 4-entry ring buffer of the most recent ACT command timestamps, used to enforce JEDEC’s tFAW constraint (no more than 4 ACT commands within any tFAW-wide rolling window, across all banks on a rank). A new ACT is permitted only when `global_cycle - faw_window[oldest] >= tFAW`.

Why this beats a simple counter: a plain “count ACTs in the last tFAW cycles” counter would need to know exactly when each contributing ACT *ages out* of the window, which requires tracking individual event timestamps anyway — a sliding window ring buffer makes that tracking explicit and minimal: only the K most recent timestamps are kept (here $K = 4$, fixed by the JEDEC constraint itself), not a full history.

Technique 9: Store-to-Load Forwarding

Industry name: Pipeline Hazard Forwarding / Bypassing

The Technique

In a pipelined system where reads and writes to the same location can be in flight simultaneously, a read that targets a location with a pending, not-yet-committed write would normally have to stall until the write commits (or worse, silently return stale data if no hazard detection exists). The standard fix — textbook terminology from CPU pipeline design — is **forwarding** (also called bypassing): detect the hazard, and route the pending write’s data directly to the read, skipping the slow commit path entirely.

Used here: the entire RAW Redirect mechanism. An incoming read is checked against all valid Write Request Buffer entries (Technique 2’s fully-associative CAM). On a full or masked-coverable hit, the read is satisfied directly from the Write Data Buffer — it never enters the Bank FSM Table scan, never touches DRAM. This is the memory-controller equivalent of a CPU forwarding a value from the execute stage directly to a dependent instruction in decode, rather than waiting for it to retire to the register file.

Why this beats stalling: without forwarding, every RAW hazard would force the read to wait for the conflicting write’s full DRAM commit latency ($CWL + BL/2 + tWR$, on the order of 15+ cycles). Forwarding collapses that into a same-cycle (or near-same-cycle) buffer read. Critically, as established during this design’s review, forwarding must satisfy *only* the read — the underlying write still has to complete its real DRAM commit independently (see the RAW Redirect & Response Hold-Forward document, §5), exactly as a CPU’s forwarded value still eventually retires to the register file even though a dependent instruction already consumed it early.

Technique 10: Cheap-Filter-Then-Expensive-Verify

Industry name: Two-Stage Filtering

The Technique

When a full check is expensive, but a cheap partial check can eliminate most non-matching candidates first, split the check into two stages: a narrow, inexpensive filter that runs on every candidate, followed by a wider, more expensive verification step that only runs on filter survivors. This pattern shows up across computing — Bloom filters ahead of real hash-table lookups, branch prediction ahead of full instruction decode, hash comparisons ahead of full-content diffing.

Used here: RAW Redirect’s two-stage check (RAW Redirect & Response Hold-Forward document, §2.2). Stage A is a cheap, binary exact-match CAM compare on {bank_group, bank, row, column} only — 22 bits, XOR-based, no mask awareness. Only if Stage A finds a row-level match does Stage B run: a byte-mask coverage check (a single 64-bit AND comparison) to determine whether the matched write actually covers every byte the read needs.

Why this beats one combined ternary check: a single-stage design would need a full ternary (mask-aware) CAM across all 32 entries in parallel — roughly double the transistor count of the binary CAM (per the CAM circuit-level reference in the RAW Redirect document, §4), since every cell needs a don’t-care mask bit in addition to the stored bit. Splitting the check means the expensive 32-way parallel array only ever needs to be binary; the mask-aware logic only runs once, on the single (if any) Stage A survivor.

Technique 11: Admission Control via Reservation

Industry name: Resource Reservation (vs. Issue-and-Retry)

The Two Strategies

When a downstream resource has limited capacity, there are two ways to handle producers that might exceed it:

Strategy	Mechanism	Cost
Issue-and-retry	Attempt the operation; if the resource is full, drop/fail and retry later.	Simple to implement, but wastes the work already done on the failed attempt, and needs a retry/backoff mechanism.
Reservation (admission control)	Check resource availability <i>before</i> committing to the operation; only proceed if a slot can be guaranteed.	Slightly more upfront bookkeeping, but guarantees no wasted work and no dropped operations.

Used here: `gate_resp_fifo_avail`. Because the Response FIFO is valid-only (no `ready` offered back to the CIF — see the field definitions document, §2.3), a read command cannot be allowed to issue unless a response slot is already guaranteed to exist when its data eventually returns. The Scheduler checks Resp FIFO availability and *reserves* a slot (recorded in the Read Request Buffer's `resp_slot` field) before the RD command is permitted to issue at all — not after, and not speculatively.

Why issue-and-retry doesn't work here: a DRAM read, once issued, cannot be cancelled — the DFI command has already gone to the PHY, and data will arrive on a fixed latency schedule regardless of whether anything downstream is ready to receive it. Retrying after the fact isn't an option; the only correct strategy is to ensure capacity is already guaranteed before the irreversible step (issuing the RD command) happens.

Technique 12: Age-Based Anti-Starvation

Industry name: Aging Counters (Fairness Arbitration)

The Technique

A pure priority scheme (always service the highest-priority class first) risks starving lower-priority entries indefinitely if higher-priority work never stops arriving. The standard fix is an **aging counter**: every waiting entry accumulates “age” each cycle it goes unserved, and once age crosses a threshold, the entry’s effective priority is boosted — either to the front of its own class, or across class boundaries entirely. This guarantees an upper bound on worst-case wait time without abandoning priority-based scheduling for the common case.

Used here: the Arbitration Logic’s two-tier escalation. `AGE_THR1` (default 64 cycles) promotes an aged entry to the head of its own class (intra-class head-of-line bypass). `AGE_THR2` (default 256 cycles) allows a sufficiently aged read to escalate *across* the RD/WR mode boundary, forcing consideration even while the Scheduler is otherwise in `WR_MODE`.

Why this beats strict priority alone: without aging, a sufficiently bursty write workload could keep `mode_state` pinned in `WR_MODE` indefinitely (so long as the write pool never drains below `WR_LOW_WM`), starving any pending read arbitrarily long. The aging mechanism provides a hard upper bound — no read waits more than `AGE_THR2` cycles before its priority is forcibly escalated, regardless of write traffic.

Technique 13: Shadow / Double-Buffered Configuration Registers

Industry name: Shadow Register Pattern

The Problem

Updating a live configuration register that's actively being read by in-flight logic (e.g. changing `tRCD` while a command is mid-flight relying on the old value) risks a torn or inconsistent read — some consumers see the old value, others see the new one, within the same logical update.

The Technique

Maintain two copies of the register: an **active** copy that all logic reads from, and a **shadow** copy that absorbs new writes. The shadow copy is only promoted to active at a well-defined safe point (e.g. when the controller is confirmed idle), so every consumer always sees a fully consistent value — never a value caught mid-update.

Used here: this pattern is the formal mechanism behind the constraint already specified in the RMC Design Specification (§5.2): Config Registers “may be updated at runtime only while the controller is idle.” A shadow-register implementation makes that constraint enforceable in hardware rather than relying purely on host-software discipline — writes land in the shadow bank; a single, controller- synchronized promotion step copies shadow to active only when an idle condition is confirmed.

Why this matters even with a software-level rule: “only update while idle” as a documented software rule is not enforced by hardware — a misbehaving or buggy host could still issue a runtime config write while the controller is active. A shadow-register implementation makes the inconsistent-read scenario *structurally impossible*, rather than merely discouraged by documentation.

Technique 14: Priority Encoder / Reduction Tree

Industry name: Parallel Reduction (Priority Encoding, Argmax Tree)

The Technique

Many parallel-compare structures produce a *vector* of candidate results (e.g. a CAM’s hit vector, one bit per entry) that must be collapsed to a single winning index. Two common reduction shapes:

- **Priority encoder:** given a bit vector, return the index of the first (or last) set bit, by a fixed positional rule. Cheap, single combinational stage for moderate widths.
- **Reduction tree (argmax):** given a vector of *values* (not just bits), pairwise-compare neighbors, keep the winner, repeat $\log_2(N)$ times until one winner remains. Used when the tie-break criterion is a value comparison rather than positional priority.

Used here, twice, for two different reasons:

1. **Priority encoder:** resolving a CAM’s `row_match_vector` (RAW Redirect, §4.4 of that document) to a single index in the ordinary case of zero or one match.
2. **Reduction tree (argmax):** resolving the rare multi-hit case (e.g. duplicate-address writes) by comparing each matching entry’s timestamp and selecting the most recent — a $\log_2(N)$ -deep tree of pairwise “compare timestamps, keep the larger, carry its index along” stages.

Why the distinction matters: a plain priority encoder (positional, e.g. “lowest index wins”) is wrong for the duplicate-address tie-break, because slot position has no relationship to recency once entries are freed and reallocated out of order. The argmax reduction tree is the correct primitive specifically because the tie-break criterion (newest write wins) is a *value* comparison, not a positional one — using the cheaper priority-encoder primitive here would silently produce wrong results on the exact case it’s meant to resolve.

Technique 15: Credit-Based Flow Control

Industry name: Credit-Based Flow Control (vs. Valid/Ready Handshake)

The Two Standard Handshake Models

Model	Mechanism	Property
Valid/Ready	Producer asserts <code>valid</code> ; consumer asserts <code>ready</code> ; transfer happens only when both are high the same cycle.	Simple, but creates a combinational dependency — the producer cannot safely produce until it has observed the consumer's <code>ready</code> , which can constrain pipelining.
Credit-Based	Consumer periodically grants “credits” representing free buffer space; producer may send freely as long as it holds unspent credits, decrementing its local credit count per send.	Decouples producer and consumer timing — the producer doesn't wait on a same-cycle signal from the consumer, only on its own credit balance, which can be checked well ahead of time.

Used here, in a deliberately asymmetric way: the Response FIFO (MC Core → CIF) is specified as **valid-only** — the CIF offers no `ready` at all (field definitions document, §2.3). This is neither pure valid/ready nor classic credit-based flow control; it's a third variant where the *producer-side* (MC Core) is solely responsible for never exceeding capacity, enforced via `gate_resp_fifo_avail` (Technique 11) acting as an internal, self-imposed credit check before a read is even allowed to issue. The CIF side is freed from ever having to assert backpressure on a signal (`dfi_rddata_valid`) that cannot itself be stalled.

Why neither pure model fits directly: classic valid/ready would require the CIF to assert `ready` in a way that could, in the worst case, propagate backpressure all the way to “don't issue this DRAM read yet” — but DRAM reads, once issued, cannot be paused or cancelled (Technique 11's reasoning). Classic credit-based flow control would normally have the *consumer* (CIF) grant credits to the *producer* (MC Core) — here, instead, the producer manages its own admission control internally, because the producer is the only side with enough lead time (at RD-issue, before the DRAM round-trip even begins) to make the decision safely.

Part II: Scalability Techniques

The techniques above describe how individual blocks in the RMC are built. The techniques below describe how the *system as a whole* grows — more MC cores, more clients, wider buses, new DRAM generations — without requiring a redesign of the blocks already specified. Each one is either already a locked decision in this design or a standing principle applied consistently across it.

Technique 16: Parameterized RTL

Industry name: Configuration-Driven Width Parameterization

The Technique

No field width, bank count, or structural depth is hardcoded into RTL. Every dimension that could plausibly change between deployments — number of banks, address field boundaries, buffer depth, counter width — is expressed as a parameter or config-register value, with the RTL written generically against that parameter (e.g. **generate for** loops over bank count, rather than 16 hand-instantiated bank FSM instances).

Used here: the RMC Design Specification states this as a foundational principle (§3.1): “no field widths are hardcoded in RTL” for address mapping. The same discipline extends to `addr_map_mode`, `col_bits`, `row_bits`, `bank_bits` in the Config Registers — the Bit Field Extractor reads these at runtime rather than having field boundaries baked into the synthesis.

Why this matters for scaling: a hardcoded design supporting exactly one DRAM generation or one DIMM configuration requires an RTL respin — new verification, new synthesis, new risk — every time the target configuration changes. A parameterized design supports the same range of configurations the parameters allow, with zero RTL changes, verified once against the parameter space rather than once per configuration.

Technique 17: Modular / Hierarchical Instantiation

Industry name: Replicate-the-Core Pattern

The Technique

Build the unit that needs to scale (here, an entire MC core) as a single, self-contained module with no hidden dependencies on being “the only instance.” Scaling up is then a matter of instantiating that module N times under a thin top-level wrapper, rather than redesigning internals to accommodate multiple instances.

Used here: the 2-to-8 MC core scaling discussion concluded that every per-core structure (Write/Read Request Buffer, Bank FSM Table, `global_cycle`, RAW Redirect CAM) stays exactly as specified regardless of core count. Only a thin core-select decode stage, living above the cores in the Address Translator, needs to grow. No MC-core internal block was redesigned to support multi-core operation.

Why this beats a monolithic multi-core-aware design: a single large design that internally tracks “which of N cores am I” couples every internal block’s correctness to the current value of N . The replicate-the-core pattern means N only exists at the wrapper level — each core’s internal logic is verified once, independent of how many copies of it will ultimately be instantiated.

Technique 18: Address-Space Partitioning at the Routing Layer

Industry name: Decode-Above-the-Unit Pattern

The Technique

When a system has multiple identical scalable units (cores, banks, channels) sharing one address space, the decision of *which unit* handles a given address is made by a single decode stage positioned *above* the units — never duplicated or re-derived inside each unit itself.

Used here: core-select bits, channel bits, and rank bits all live exclusively in the CIF-side Address Translator, stripped off *before* a request reaches any specific MC core (multi-core scaling discussion). No MC-core-internal block ever needs to know how many total cores exist or which core index it is — it only ever operates on its own local 32-bit address slice.

Why this beats per-unit address awareness: if each core had to decode its own core-select bits, every core would need updating whenever the system-level core count or address width changed — exactly the coupling Technique 17 is designed to avoid. Centralizing the decode means growing the system (more cores, wider addresses) only touches one block, not every replicated unit.

Technique 19: Interface Width Over-Provisioning

Industry name: Forward-Sized Interfaces

The Technique

Size an external-facing interface (a bus width, an address field) for the eventual ceiling of what the system will need to support, even if only a subset of that width is “live” in the current deployment. Unused high-order bits are tied to a fixed value (typically 0) or driven by configuration, rather than the interface itself needing to widen later.

Used here: the AXI address port is sized to the eventual 44-bit ceiling (multi-core, multi-rank, multi-channel future target) even though only the low 32 bits are live for the current single-rank, single-channel, 4 GB configuration. The high bits tie to 0 (core-select = 0) until multi-core support is actually built.

Why this beats widening later: an interface width change is one of the most invasive changes possible in a hardware design — every block touching that signal, every testbench, every downstream consumer needs updating. Over-provisioning the width up front costs nothing functionally (unused bits are free in simulation and cheap in synthesis) and avoids ever needing that invasive change.

Technique 20: Shared Resource Pooling

Industry name: Pooled vs. Per-Client Allocation

The Technique

When a resource (buffer entries, CAM slots) must serve multiple concurrent clients, allocate from a single shared pool sized for *expected concurrent load*, rather than reserving a fixed per-client allocation sized for worst-case simultaneous use by every client at once.

Used here: the Write/Read Request Buffers are 32–64 entries **total**, shared across up to 32 AXI clients — not 32 entries *per client* (which was explicitly considered and rejected during the outstanding-ID sizing discussion, since 32-per-client would have required 1024 entries for worst-case occupancy).

Why this beats per-client reservation: per-client allocation scales linearly with client count even though real traffic is rarely “every client maxed out simultaneously.” A shared pool scales sublinearly — the pool size is chosen for sustainable average concurrent load, with fairness (Technique 12 from Part I) absorbing the unevenness, rather than the pool size itself needing to grow with every additional client.

Technique 21: Fairness-Based Pool Sharing

Industry name: Aging in Place of Static Reservation

The Technique

When a shared pool (Technique 20) is contended by more clients than it can simultaneously serve at full depth each, use a fairness mechanism (aging, round-robin) to guarantee bounded wait time per client — rather than statically partitioning the pool into fixed per-client reservations that would otherwise be needed to guarantee the same fairness property.

Used here: the same age-counter mechanism from Part I, Technique 12 ([AGE_THR1](#)/[AGE_THR2](#)) is what makes Shared Resource Pooling (Technique 20) safe to scale to more clients without redesign — adding clients only means retuning the age thresholds if needed, never restructuring the pool itself into per-client partitions.

Why this matters distinctly from Technique 20: pooling alone doesn't guarantee fairness — a naive shared pool could still let one aggressive client starve others indefinitely. Pairing the pool with an aging mechanism is what allows the pool to scale to *more* clients without that risk growing, since the fairness guarantee is a property of the arbitration logic, not the pool size.

Technique 22: Throttled Admission

Industry name: Meter-Don't-Burst Pattern

The Technique

When an external interface can present far more work in a single transaction than internal structures are sized to absorb at once (a large AXI burst vs. a modest request buffer), don't accept the entire transaction into internal storage up front. Instead, hold the transaction's remaining work in a small, fixed-size context at the boundary, and admit it into internal structures incrementally — one unit at a time, exactly as capacity frees up.

Used here: the 256-beat AXI burst problem (up to 256 sub-requests from a single client's maximum-size transaction, against a 64-entry shared buffer). The fix adopted was a throttled Burst Splitter: a small per-client burst context (`base_addr`, `remaining_beats`, `axi_tag`) metering sub-requests into the shared pool one at a time, rather than the buffer needing to be sized for worst-case burst length.

Why this beats sizing the buffer to match the interface: the interface ceiling (256 beats $\times$ 32 clients) and the internal buffer depth (64 entries) are fundamentally different scaling problems. Trying to size the buffer to match the interface's worst case is a losing battle (computed directly: 8192 potential outstanding sub-requests vs. any realistic buffer size). Metering decouples the two — the interface can scale its own ceiling independently, since the buffer is never asked to absorb more than it's sized for at any given instant.

Technique 23: Single Clock-Domain-Crossing Boundary

Industry name: Centralized CDC

The Technique

When two clock domains must communicate, route *all* cross-domain traffic through one well-verified crossing structure (a pair of async FIFOs), rather than allowing ad hoc crossings to appear wherever convenient as the design grows.

Used here: the RMC Design Specification states this explicitly (§5.1): the Async FIFO CDC bar is “the sole crossing point” between the AXI clock domain and the MC clock domain. Every block upstream (CIF, Burst Splitter, Address Translator, ROB) runs on AXI clock; every block downstream (Bank FSMs, Scheduler, Maintenance Engine, DFI) runs on MC clock — with no exceptions.

Why this matters for scaling: CDC logic is among the hardest hardware correctness problems to verify (metastability, reconvergence, clock-domain-crossing-specific lint rules). Confining it to one boundary means scaling the design — adding more clients, more cores, more buffer depth on either side — never multiplies the *number* of CDC crossings that need independent verification; it only changes the data volume flowing through the one crossing already proven correct.

Technique 24: Bounded Associativity

Industry name: Throttle-the-Input Before You Grow-the-CAM

The Technique

Before committing to a fully-associative structure (Part I, Technique 2) at a given depth N , explicitly check whether N can plausibly need to grow without bound as the system scales. If so, the correct response is to bound or throttle the *input* to that structure (Technique 22) rather than scaling the CAM itself, since CAM cost grows linearly (or worse) with depth.

Used here: the outstanding-ID sizing discussion explicitly walked this decision tree — 64-entry CAM is affordable, 256-entry is expensive (roughly $4\times$ the comparator area), and the true worst case (8192, from $32 \text{ clients} \times 256 \text{ beats}$) is not a CAM-sizing problem at all, it's an admission-control problem, resolved by Technique 22 instead of by growing the CAM.

Why this is a distinct technique from Throttled Admission: it's specifically the *decision rule* for recognizing when a fully-associative structure's natural depth requirement has outgrown what's affordable, and routing the fix to the input side rather than reflexively scaling the expensive structure. Without this explicit check, the default instinct ("just make the CAM bigger") leads directly into the area explosion this technique exists to avoid.

Technique 25: Peer-FSM Extensibility

Industry name: Add-a-Peer, Not Add-a-Special-Case

The Technique

When new background functionality needs to be added to a system (a new maintenance operation, a new periodic housekeeping task), implement it as a new, independent FSM that writes into *existing* shared state structures through the same interface other peer FSMs already use — rather than adding a new special-case branch inside the central controller/scheduler itself.

Used here: the Maintenance Engine placement discussion (Model A) established that Refresh Scheduler, ZQcal, RFM, and Power Management are all peer sub-FSMs, each independently writing into the existing Bank FSM Table’s `ref_pending`-style flags. RFM was explicitly noted as “the third instance of the same pattern” as REF — not a structurally new code path inside the Scheduler.

Why this beats special-casing inside the Scheduler: if every new background feature required new Scheduler-internal logic, the Scheduler’s complexity would grow with feature count indefinitely, and each addition would risk destabilizing previously-verified scheduling behavior. The peer-FSM pattern means the Scheduler’s Stage 2 scan logic is unchanged by adding RFM, ECS, or any future maintenance feature — it already knows how to consume a pending-flag-style input, regardless of which FSM produced it.

Technique 26: Single Global Timebase

Industry name: One Clock Reference, Many Comparators

The Technique

When many independent constraints each need their own notion of “how much time has passed,” share a single free-running counter as the universal time reference, rather than giving each constraint its own independent timer.

Used here: this is Part I Technique 3 restated specifically through a scaling lens — `global_cycle` is the sole time reference for every deadline field in the system. Adding a new timing constraint (a new JEDEC parameter, a new maintenance operation’s recovery time) costs exactly one new deadline field and one new comparator; it never requires a new independent counter.

Why this matters distinctly for scaling: a per-constraint timer design means every new constraint added as the system grows (new DRAM generation, new feature) requires new counter hardware. The shared timebase means growth only ever costs comparator logic — cheaper, and critically, all constraints remain comparable against each other on the same clock reference, which matters when, e.g., a newly-added RFM deadline needs to be compared for priority against an existing REF deadline.

Technique 27: Tiered Hierarchical Arbitration

Industry name: Staged Arbitration, Not Flat N-Way

The Technique

When a decision depends on many factors of differing priority and scope (global overrides, mode selection, per-unit eligibility, fine-grained tie-breaking), structure the decision as a sequence of narrow stages — each handling a bounded comparison set — rather than one flat arbitration evaluating every factor against every other factor simultaneously.

Used here: the Cmd Scheduler’s five stages (priority pre-check, RD/WR mode arbitration, per-bank go/no-go scan, bank-group-aware selection, command emission) is exactly this structure. Stage 0’s refresh/ZQ override never needs to be compared against Stage 3’s bank-group tie-break logic directly — each stage only compares within its own bounded scope.

Why this scales better than flat arbitration: a flat scheme comparing every possible (bank, command-type, priority-class) combination against every other grows combinatorially as banks, command types, or priority levels are added. Tiered arbitration means adding a new priority level (Technique 24’s peer-FSM pattern, for instance) only adds a check at the appropriate existing stage, not a combinatorial expansion of the whole arbitration network.

Technique 28: Rate-Decoupling Buffers

Industry name: Buffer-the-Mismatch, Not Match-the-Clocks

The Technique

When two blocks naturally operate at different throughput rates (not necessarily different clock domains — could be different processing latencies on the same clock), insert a buffer sized for the rate mismatch between them, rather than requiring either block's internal pipeline depth or timing to match the other's.

Used here: this is the general form of the CDC FIFOs (Technique 23 is the clock-domain-specific case). The same principle applies wherever a fast producer feeds a slower consumer in this design — the Resp FIFO's hold-and-forward mechanism (RAW Redirect document, §6) buffers the rate mismatch between two independent response sources (RAW hits, DRAM returns) and the single-port Resp FIFO drain rate, without requiring either source to internally throttle itself to match the FIFO's instantaneous capacity.

Why this is broader than just CDC: clock-domain crossings are the most obvious case, but rate mismatches occur within a single clock domain too (two combinational sources both able to fire the same cycle, draining into one sequential resource). The general lesson — buffer the mismatch rather than constrain the producers — is what generalizes as the system scales and more independent sources are added.

Technique 29: Configuration-Time Capability Negotiation

Industry name: Init-Time Profile Selection, Not Compile-Time Branching

The Technique

When a system must support multiple variants of an external protocol or device generation, select the active behavior via a configuration value read at initialization (a profile, a mode register), rather than compiling separate RTL variants for each supported generation.

Used here: the `DDR_VER` register selects which JEDEC timing gate set and command sequence applies, read by the Init FSM at boot. The Init FSM's command sequence and inter-command delays come from Config Registers rather than being hardcoded per DDR generation — the “Reconfigurable” in this design's name refers directly to this technique.

Why this beats compile-time variants: compile-time branching (separate RTL builds per supported DRAM generation) multiplies verification effort by the number of supported variants and risks divergence between variants over time. Runtime profile selection means one RTL build is verified once across its configuration space, and adding support for a new DRAM generation is a configuration-profile addition, not a new RTL target.

Technique 30: Depth-Bounded Graceful Degradation

Industry name: Bound the Resource, Let Logic Absorb Pressure

The Technique

For any resource whose ideal size scales with an input the system doesn't fully control (client count, traffic burstiness), deliberately cap that resource at an affordable depth, and rely on control logic (fairness, backpressure, throttling) to keep behavior correct — if degraded — under loads that exceed the resource's comfortable capacity, rather than attempting to size the resource for unbounded worst-case input.

Used here: the Write/Read Request Buffers stay at 64 entries even as supported client count grows from 8 to 32. The explicit design stance — confirmed during the outstanding-ID discussion — is that buffer depth does not scale with client count; age-based fairness (Technique 21) and throttled admission (Technique 22) absorb the resulting pressure instead.

Why bounding is a deliberate choice, not a limitation: the alternative — sizing for true worst case — was shown to be infeasible (thousands of entries for full 32-client saturation). Explicitly bounding the expensive resource and accepting that the system's behavior under extreme load is “fair but throttled” rather than “unboundedly fast” is the only scalable choice once worst-case sizing is ruled out.

Technique 31: Event-Driven Flag Propagation

Industry name: Flat Flag Reads, Not Deep Query Chains

The Technique

When one block (a consumer) needs to know about conditions tracked by several independent producer blocks, have each producer expose its condition as a simple, flat flag the consumer reads directly — rather than having the consumer query each producer’s internal state through a chain of nested lookups.

Used here: `ref_pending`, `ref_urgent`, and equivalent flags from each Maintenance Engine sub-FSM are read directly by the Scheduler’s Stage 0/2 logic as flat bits — the Scheduler never needs to reach into the Refresh Scheduler’s internal credit-counter state or the RFM FSM’s internal RAA tracking to determine eligibility.

Why this matters as more producers are added: a query-chain design (Scheduler reaching into each sub-FSM’s internals individually) means adding a new sub-FSM (Technique 24) requires the Scheduler to learn a new query interface for that specific FSM. Flat flag propagation means every sub-FSM exposes the same kind of simple output regardless of its internal complexity — the Scheduler’s read logic doesn’t grow more complex as more producers are added behind the same flag interface.

Technique 32: Symmetric Per-Unit Design

Industry name: No Special-Cased “Unit Zero”

The Technique

When a design contains N structurally identical units (banks, cores, clients), ensure every unit — including the first one built — follows exactly the same template, with no logic that implicitly assumes “this is the only/first/special instance.” This avoids the common scaling trap where the first instance accumulates convenience shortcuts that then have to be retrofitted into every subsequent instance.

Used here: every bank is a uniform row in the same Bank FSM Table (Technique 18’s replicate pattern at the bank level, not just the core level) — there is no “bank 0 is handled specially” logic anywhere in the field definitions. Equally, the multi-core wrapper (Technique 18) treats every MC core instance identically; no core is designated as a primary or special instance with extra responsibilities.

Why this matters specifically when scaling from 1 to N: a design first built for a single instance (one bank, one core) is especially prone to this trap, since there’s no immediate pressure to generalize until a second instance is actually needed. Enforcing symmetry from the first instance onward means scaling to N is purely a matter of instantiation count, never a retrofit of logic that quietly assumed $N = 1$.

Technique 33: Explicit Interface Contracts

Industry name: Documented Boundaries Over Defensive Logic

The Technique

At every boundary where one part of a system hands data or control to another, write down the exact contract — what’s guaranteed, what’s excluded — rather than leaving it implicit. Internal logic can then be built to rely on the contract holding, instead of defensively handling cases the contract already rules out.

Used here: the AXI Feature Decisions table (RMC Design Specification, §4.1) is exactly this — INCR-only, no narrow transfers, no unaligned transfers, no exclusive access, explicitly written down with rationale for each exclusion. Every block downstream of the CIF is built assuming these exclusions hold, rather than containing defensive logic for WRAP bursts or unaligned addresses that the contract has already ruled out.

Why this matters for scaling: as the system grows (more clients, wider buses), the question “does this still work?” only requires checking whether the documented contract still holds for the new scale — not re-auditing every internal block for untested edge cases the contract was always supposed to exclude. Implicit assumptions, by contrast, tend to surface as bugs only once scaling stresses a path nobody remembered to check.

Technique 34: Generalized Reservation-Before-Commitment

Industry name: Check-Then-Commit at Every Scaling Boundary

The Technique

Part I, Technique 11 (Admission Control via Reservation) describes one instance of a more general pattern: at any boundary where an irreversible action depends on downstream capacity, verify that capacity is available *before* taking the action, not after. As a system scales and new boundaries of this kind appear (new cross-core resource sharing, new shared structures), the same check-then-commit discipline should be applied rather than re-deriving an ad hoc solution per boundary.

Used here, as a standing principle beyond the one FIFO case: `gate_resp_fifo_avail` is the concrete instance already built. The pattern itself — reserve before you commit to an irreversible action — is flagged as the template to reapply if/when multi-core scaling introduces new shared resources (e.g. a cross-core arbitration point) that would otherwise need their own bespoke backpressure scheme.

Why naming this separately from Technique 11 matters: Technique 11 documents *what was built*. This technique documents *the design stance to apply again* the next time a similar boundary appears during scaling — distinguishing a one-off solution from a reusable discipline is itself part of designing for growth.

Technique 35: Self-Describing Configuration

Industry name: Explicit Version/Width Fields Over Inferred Structure

The Technique

Store structural metadata — field widths, active protocol version, enabled feature set — as explicit, readable configuration register values, rather than having RTL or external tooling infer that structure from indirect signals (e.g. address value ranges, signal timing patterns).

Used here: `addr_map_mode`, `bg_bits`, `bank_bits`, `DDR_VER` are all explicit, directly-readable configuration fields describing the system's current structure. A debug tool, testbench, or verification environment can read these registers directly to learn the active configuration, rather than needing to infer it from observed address patterns or timing behavior.

Why this matters as the system scales: every new configuration the system needs to support (Technique 26's capability negotiation) is only useful if the configuration is actually *discoverable* — by debug infrastructure, by verification environments, by anyone reasoning about the system after the fact. Self-describing configuration means tooling built against this design doesn't need updating every time a new configuration profile is added; it can always just read the registers.